

Roberto Pinto Gutiérrez

Actividad 1.4

DWES 26/27:

Tema 1 : Arquitectura y herramientas de programación Web :

Ejercicio 1 :

La arquitectura de un servidor web varía en función de la plataforma sobre la que se vaya a utilizar y las capacidades que se requieren. Algunos de los servidores web más conocidos son Apache, de la Apache Foundation, o Internet Information Server de Microsoft.

Proponemos que el alumno busque la última versión de estos servidores y describa tanto su arquitectura básica como los mecanismos de extensión que ofrecen para el soporte de diferentes tecnologías de programación en entorno del servidor.

1.1. Apache HTTP Server

La última versión estable de Apache HTTP Server en el momento de realizar este trabajo es la 2.4.68, publicada por la Apache Software Foundation en junio de 2026 dentro de la rama 2.4.x, vigente desde 2012 y recomendada frente a todas las versiones anteriores por incluir todas las correcciones de seguridad y mejoras acumuladas.

La arquitectura de Apache se organiza en tres capas:

- **Núcleo (core):** gestiona la configuración, la comunicación en red de bajo nivel y define los puntos de enganche sobre los que se construyen el resto de funcionalidades.
- **Apache Portable Runtime (APR) y APR-Util:** capa de abstracción que permite que el mismo código funcione en distintos sistemas operativos (Linux, Windows, BSD, etc.), ocultando las diferencias de las llamadas al sistema.
- **Módulos de procesamiento múltiple (MPM, Multi-Processing Modules):** definen cómo se atienden las conexiones concurrentes. Los más usados son *prefork*, *worker* y *event*, que es el MPM recomendado en los despliegues actuales de alto tráfico.

Mecanismos de extensión de Apache

Apache es, por diseño, un servidor mínimo al que se le añaden capacidades mediante módulos. Existen dos formas de incorporarlos:

- **Módulos estáticos:** se compilan junto con el binario httpd. Ofrecen algo más de rendimiento pero obligan a recompilar el servidor para añadir o quitar funcionalidad.
- **Módulos dinámicos (DSO, Dynamic Shared Objects):** se cargan en tiempo de ejecución con la directiva `LoadModule` y el módulo `mod_so`, sin necesidad de recompilar Apache. Es el mecanismo habitual hoy en día porque facilita el mantenimiento.

Para el soporte de lenguajes de programación en el entorno del servidor, Apache ofrece varias vías, que han ido cambiando con el tiempo:

- **Módulos de lenguaje embebidos** en el proceso del servidor, como `mod_php` (que integraba directamente el intérprete de PHP) o `mod_perl`. Este enfoque es cada vez menos habitual porque acopla el ciclo de vida del lenguaje al del servidor.
- **CGI y FastCGI**: el servidor lanza (CGI) o mantiene abierto (FastCGI) un proceso externo que ejecuta el script y devuelve la respuesta por un canal estándar. Es independiente del lenguaje.
- **Proxy inverso hacia un proceso gestor de lenguaje**, como `mod_proxy_fcgi` combinado con PHP-FPM (FastCGI Process Manager), que es el modelo recomendado actualmente para PHP, o `mod_proxy_wstunnel/mod_proxy` hacia aplicaciones Node.js, Python (Gunicorn/uWSGI) o Java.
- **mod_wsgi**, específico para desplegar aplicaciones Python que cumplen el estándar WSGI directamente dentro de Apache.

1.2. Internet Information Services (IIS)

IIS es el servidor web de Microsoft integrado en Windows Server. La versión vigente es IIS 10.0, incluida en Windows Server 2016, 2019, 2022 y 2025, que ha ido recibiendo mejoras incrementales (como el soporte de HTTP/2 y HTTP/3) sin cambiar el número de versión principal.

Su arquitectura se apoya en un modelo híbrido entre el modo kernel y el modo usuario:

- **HTTP.sys**: controlador en modo kernel que recibe todas las peticiones HTTP/HTTPS que llegan al servidor, gestiona la caché de respuestas y las reparte a los procesos de trabajo correspondientes. Al residir en el kernel, evita cambios de contexto innecesarios y mejora el rendimiento y la estabilidad frente a caídas de aplicaciones concretas.
- **WWW Publishing Service (W3SVC) y WAS (Windows Process Activation Service)**: en modo usuario, coordinan los grupos de aplicaciones (application pools) y arrancan/detienen los procesos de trabajo bajo demanda.
- **Procesos de trabajo (w3wp.exe)**: cada grupo de aplicaciones se ejecuta de forma aislada en su propio proceso, de modo que un fallo en una aplicación no afecta a las demás.
- **Canalización de módulos (pipeline)**: las peticiones atraviesan una serie de módulos nativos e integrados (autenticación, compresión, caché, etc.) organizados en el modo integrado (Integrated Mode), introducido en IIS 7, que unificó el pipeline de ASP.NET con el del servidor.

Mecanismos de extensión de IIS

- **Módulos nativos**: componentes escritos en C++ que se registran en el pipeline de IIS y pueden interceptar cualquier fase del procesamiento de la petición.

- **Módulos administrados:** componentes .NET que implementan interfaces como IHttpModule o IHttpHandler; se ejecutan en modo integrado junto con los módulos nativos.
- **ISAPI (Internet Server API):** interfaz heredada basada en DLL para extensiones y filtros; sigue siendo compatible por motivos de continuidad, aunque hoy se recomiendan los módulos nativos o administrados.
- **ASP.NET Core Module (ANCM):** reenvía las peticiones a un proceso independiente de Kestrel (el servidor propio de ASP.NET Core), permitiendo alojar aplicaciones .NET modernas multiplataforma.
- **FastCGI:** mecanismo utilizado para ejecutar lenguajes externos como PHP dentro de IIS, de forma equivalente al FastCGI de Apache.

Ejercicio 2 :

Los lenguajes del entorno del servidor presentados han ido evolucionando históricamente incluyendo cada vez más funcionalidades. Proponemos que el alumno elija uno de esos lenguajes y realice una descripción detallada de su evolución, indicando cuáles son las influencias recibidas de otros lenguajes y sobre qué otros lenguajes ha influido.

De los lenguajes de programación en el entorno del servidor, se ha elegido **PHP** por ser uno de los más utilizados en el desarrollo web y por tener una evolución especialmente ilustrativa de cómo un lenguaje pensado inicialmente como una simple colección de utilidades acaba convirtiéndose en un lenguaje de propósito general orientado a objetos.

2.1. Orígenes: PHP/FI (1994-1997)

PHP nace en 1994 de la mano de Rasmus Lerdorf como un conjunto de scripts en C, llamado "Personal Home Page Tools", pensado para llevar el control de las visitas a su propia página. Poco después incorporó un intérprete y pasó a llamarse PHP/FI (Personal Home Page / Forms Interpreter), añadiendo soporte para formularios HTML y acceso a bases de datos mSQL.

2.2. PHP 3 y PHP 4: nacimiento del lenguaje moderno

En 1997, Andi Gutmans y Zeev Suraski reescribieron el intérprete desde cero al comprobar que el motor original era demasiado limitado; el resultado se publicó como PHP 3 (1998), la primera versión que se pareció al PHP actual, con una sintaxis coherente y soporte multiplataforma. Para PHP 4 (2000) ambos desarrolladores crearon un nuevo motor, el Zend Engine, mucho más eficiente, que introdujo el sistema de buffering de salida y una gestión de sesiones nativa.

2.3. PHP 5: la orientación a objetos

PHP 5 (2004) incorporó el Zend Engine 2 y una reformulación completa del modelo de objetos (visibilidad, interfaces, clases abstractas, excepciones), inspirado directamente en el modelo de clases de Java, así como soporte para SQLite y la extensión PDO para acceso homogéneo a bases de datos. La versión 5.3 (2009) añadió espacios de nombres (namespaces) y funciones anónimas (closures), dos características tomadas de lenguajes como Python y JavaScript respectivamente.

2.4. PHP 7: el salto de rendimiento

PHP 7 (2015) sustituyó el motor por el Zend Engine 3, resultado del proyecto "PHPNG" (PHP Next Generation), que reorganizó las estructuras internas de datos y duplicó o triplicó el rendimiento respecto a PHP 5 sin cambiar apenas la sintaxis del lenguaje. También incorporó el tipado escalar opcional en parámetros y valores de retorno, así como el operador null-safe "??".

2.5. PHP 8 y versiones posteriores: hacia un lenguaje tipado y funcional

PHP 8 (2020) añadió un compilador JIT (Just In Time), tipos unión, la expresión match, los atributos (equivalentes a las anotaciones de Java) y el operador nullsafe "?->". Las versiones siguientes han continuado esa línea: PHP 8.1 introdujo los enums y las propiedades de solo lectura (readonly); PHP 8.2 añadió las clases readonly completas; PHP 8.3 sumó tipado explícito en constantes de clase; PHP 8.4 (2024) incorporó los property hooks. La versión más reciente, **PHP 8.5**, se publicó en noviembre de 2025 e incluye el operador pipe (|>) para encadenar funciones de izquierda a derecha, una extensión nativa para el tratamiento de URIs y las funciones array_first()/array_last(), entre otras novedades; a lo largo de 2026 sigue siendo la versión estable de referencia mientras se prepara PHP 8.6 para finales de año.

2.6. Influencias recibidas

- **Perl:** PHP hereda de Perl la sintaxis de variables con el símbolo \$, las expresiones regulares y la orientación a procesamiento de texto propia de los primeros scripts CGI.
- **C:** la sintaxis general de bloques, operadores y estructuras de control (if, for, while) procede directamente de C, lenguaje en el que además está escrito el propio intérprete de PHP (Zend Engine).
- **Java:** el modelo de clases, interfaces y excepciones introducido en PHP 5 se inspiró en el de Java.
- **Python y JavaScript:** los espacios de nombres y las funciones anónimas (closures) incorporados en PHP 5.3 recogen ideas ya asentadas en estos lenguajes.

2.7. Influencia sobre otros lenguajes y tecnologías

- **Hack:** Facebook creó en 2014 el lenguaje Hack como una variante de PHP con tipado estático opcional, ejecutada sobre su propia máquina virtual HHVM, precisamente para resolver limitaciones de rendimiento y de tipado que tenía PHP en aquel momento.

- **Frameworks y CMS:** el propio éxito de PHP impulsó la aparición de gestores de contenidos como WordPress, Drupal o Joomla, y de frameworks como Laravel o Symfony, que han influido a su vez en cómo se diseñan otros frameworks de servidor en lenguajes distintos (por ejemplo, el patrón de enrutado y ORM de Laravel es una referencia habitual para frameworks equivalentes en Node.js o Python).
- **Motores de plantillas:** la mezcla de código y HTML propia de los primeros scripts PHP sirvió de modelo para sistemas de plantillas posteriores en otros lenguajes de servidor (JSP en Java, ERB en Ruby, EJS en Node.js).

Ejercicio 3 :

Al igual que los servidores cuentan con extensiones, las herramientas de programación web también pueden ser extendidas. Busque información sobre uno de los editores mencionados y amplíe su información indicando los métodos en los que pueden ser extendidos:

Del mismo modo que los servidores web se amplían mediante módulos, los editores de código utilizados en el desarrollo web también están diseñados para ser extendidos. Se ha elegido **Visual Studio Code** (VS Code), de Microsoft, por ser en la actualidad uno de los editores más utilizados en el desarrollo de aplicaciones web.

3.1. Arquitectura de extensibilidad

VS Code está construido sobre Electron (que combina Chromium y Node.js) y separa claramente dos procesos: el proceso principal, donde se ejecuta la interfaz, y el proceso de extensiones (Extension Host), donde corre el código de terceros de forma aislada. Esta separación evita que una extensión mal programada bloquee la interfaz del editor y es la base de todo su sistema de extensión.

3.2. Métodos de extensión

- **Extension API (API de extensiones):** es el mecanismo principal. Las extensiones se escriben en TypeScript o JavaScript sobre Node.js y se empaquetan como paquetes .vsix, que pueden instalarse desde el Marketplace de Visual Studio Code o de forma manual. A través de esta API se pueden añadir comandos, vistas laterales, resaltado de sintaxis, formateadores, depuradores o paneles propios (webviews basados en HTML/CSS/JS).
- **Language Server Protocol (LSP):** protocolo abierto, creado por Microsoft junto con la comunidad, que separa la lógica de análisis de un lenguaje (autocompletado, comprobación de errores, "ir a definición") en un proceso independiente llamado servidor de lenguaje. El editor solo necesita implementar un cliente ligero que hable

ese protocolo, lo que permite reutilizar el mismo servidor de lenguaje en otros editores compatibles con LSP (Neovim, Sublime Text, entre otros).

- **Debug Adapter Protocol (DAP):** protocolo equivalente al LSP pero para depuradores; permite conectar VS Code con el depurador de un lenguaje concreto (Xdebug para PHP, el depurador de Node.js, GDB, etc.) sin que el editor tenga que conocer los detalles internos de cada depurador.
- **Temas e iconos:** paquetes declarativos, normalmente en JSON, que permiten cambiar el aspecto visual del editor (colores, tipografía de iconos de archivos) sin necesidad de escribir código.
- **Snippets:** fragmentos de código reutilizables definidos en ficheros JSON, que se insertan al escribir una palabra clave y pulsar Tab.
- **Task Providers e integración de herramientas externas:** las extensiones pueden registrar tareas (compilación, linters, empaquetadores) que se ejecutan desde la paleta de comandos y muestran su salida en el panel de terminal integrado.
- **Remote Development:** extensiones especiales (Remote-SSH, Dev Containers) que permiten que el Extension Host se ejecute en una máquina remota o en un contenedor Docker, mientras la interfaz sigue funcionando en local.

Ejercicio 4 :

Google Web Server.

Google Web Server (GWS) es el servidor web propietario que Google utiliza internamente para servir el buscador y otras propiedades de su ecosistema, como se puede comprobar observando la cabecera HTTP "Server: gws" que devuelven sus páginas.

A diferencia de Apache o IIS, GWS no es un producto público ni de código abierto, y la información disponible sobre él es muy escasa porque Google lo considera una pieza estratégica y sensible de su infraestructura. Los datos que se conocen, procedentes de comunicados públicos y comparativas de mercado, son los siguientes:

- Está desarrollado íntegramente en **C++**.
- Se ejecuta sobre una versión de **Linux (basada en Debian)** fuertemente modificada y optimizada por Google para sus centros de datos.
- No se distribuye ni se vende: su uso está restringido a la infraestructura interna de Google.
- Ha llegado a servir en torno al **13% de todos los sitios web del mundo** (medido en 2010) debido al enorme volumen de tráfico que gestiona Google, y en 2015 se situaba como el cuarto servidor web más usado de Internet, por detrás de Apache, nginx e IIS.

En un comunicado de una oficina de Google en Chicago del año 2011, la empresa describió al equipo de GWS como el encargado de construir y mejorar la infraestructura propietaria de servicio web que da soporte a la búsqueda de Google y a otras muchas propiedades

relacionadas, señalando que casi cualquier cambio visible en google.com pasa por ese equipo.

Por su carácter cerrado, no es posible describir su arquitectura interna, sus mecanismos de extensión ni sus versiones con el mismo detalle que en el caso de Apache o IIS; lo que trasciende es que Google prioriza el rendimiento y el control absoluto sobre cada capa de su infraestructura antes que ofrecer un producto reutilizable por terceros, algo coherente con la escala a la que opera la compañía.